ASignX - Detailed System Report

22 October 2026 | Asia/Singapore

System architecture, machine learning, data quality, recognition results, browser integration and development priorities.

Prepared from repository source, saved experiment records, inspected checkpoints, a fresh comparison of the two available static classifiers, and a fresh software test run. Technical terms are explained where they first matter and collected in the glossary.

Executive assessment

ASignX is a hand-landmark recognition prototype. It combines Google's pretrained MediaPipe hand detector with the project's own small neural networks. The detector estimates hand geometry; the ASignX classifiers assign a sign label to that geometry. The default desktop launcher uses a static classifier for 24 alphabet poses and a separate movement classifier for J and Z. This establishes an alphabet recognition workflow, rather than general sign-language translation.

The current main dataset contains 159,534 static samples, 24 classes and 1,648 session identifiers. Today's data contain 1,800 more M samples and nine more session identifiers than the previously reviewed 157,734-row snapshot. The new samples are Left-hand M records. A completed manual training run now exists beside the two default checkpoints. Earlier reports saying there were only two saved models or that the current CSV had 157,734 rows describe an earlier state.

The manual model has a saved test macro F1 of 74.23% and accuracy of 59.32%. Macro F1 is the average of the individual sign classes' F1 scores, giving every class equal weight. This score is on a newly generated split, so it cannot be compared directly with the default model's historical cleaned-test macro F1 of 63.07%. A fresh evaluation of the default model on the manual run's test rows gives 74.84% macro F1 and 61.79% accuracy. However, 947 of those test rows share sessions with the default model's old training set, and 356 share sessions with its old validation set. This comparison is descriptive and does not establish an independent winner.

The main unresolved problem is evidence of generalization: whether recognition remains accurate for people, source datasets and recording conditions that were absent from development. Data integrity protections and software tests are useful and substantial, but cannot substitute for that evidence. The dynamic model's 100% score covers only two test clips; it does not establish dependable movement recognition.

This report preserves the earlier reports' substantive scope: model explanations, extraction and cleaning, evaluation, architecture alternatives, body coverage, new signs, bottlenecks, ASignLearn, ASignLive and browser deployment. The earlier conversation itself is not available in this session; the saved reports and development records provide the recoverable requirements and history.

Contents

1. Scope, evidence and terminology . . . 3
2. What the system currently supports . . . 4
3. End-to-end architecture . . . 4
4. Landmark data and what the representation loses . . . 5
5. Preprocessing, normalization and handedness . . . 6
6. Static model: a kinematic residual MLP . . . 6
7. Dynamic model: temporal convolution and attention . . . 7
8. How training works . . . 8
9. Current dataset inventory and today's changes . . . 9
10. Splitting, leakage and benchmark changes . . . 10
11. Extraction and cleaning workflow . . . 11
12. Source recovery and the M corrections . . . 11
13. Understanding recognition metrics . . . 12
14. Current static results and their interpretation . . . 12
15. Per-class results and targeted diagnosis . . . 13
16. Dynamic results and shortcut learning . . . 15
17. Live confidence, motion routing and stabilization . . . 15
18. ASignLearn: coaching and interpretation . . . 16
19. ASignLive and the current web SDK . . . 16
20. Browser export, performance and deployment . . . 17
21. Are hands enough, and which models make sense? . . . 18
22. Scaling the vocabulary and adding signs . . . 19
23. Current tools, artifacts and development history . . . 20
24. Priorities, completion criteria and limits . . . 21
25. Plain-language ML glossary . . . 22
26. Reproducibility and evidence index . . . 23

1. Scope, evidence and terminology

Runtime migration update, 22 October 2026: active Python workflows now use the project's Python 3.14.7 environment. Reporting, packaging and ONNX dependencies are installed there. All 114 tests passed with seven warnings in 29.95 seconds after the export adjustment. Fixed-input checks produced identical Python 3.11/3.14 logits and predicted labels for all three saved checkpoints; exported ONNX outputs agreed within 0.000004. These checks cover 128 static samples and one sequence, not a new accuracy benchmark. One source-image extraction produced the same handedness and one detected hand in each environment, with a maximum coordinate difference of about 0.000365. This limited check does not establish detector equivalence across a dataset. The original training/evaluation history below is preserved. Migration evidence and current operational commands are documented in docs/python314-migration.md and output/python314-migration/.

The review date is 22 October 2026 in Asia/Singapore. A current fact means a property inspected in this workspace or measured during this report. A historical result means a saved measurement tied to an older dataset or split. A recommendation is a proposed next step, not a capability demonstrated by the current application.

The report uses three distinct static benchmarks: the original 159,987-row dataset, the 157,734-row cleaned snapshot after the M corrections, and today's 159,534-row dataset used by the manual training run. Each has a different SHA-256 identity. SHA-256 is a content fingerprint: changing the file changes its fingerprint. It helps identify the exact input to an experiment; it does not certify that the labels are correct.

A model checkpoint is the saved learned weights together with information needed to rebuild the network. Weights are adjustable numerical values learned during training. A manifest is a structured record describing a dataset or experiment; a split manifest records which rows belong to training, validation and testing. These records must agree with the dataset they describe.

Evidence is local unless an external reference is explicitly cited. Main references are main.py, asignx/, ml/, web_client/asignx_client_sdk.ts, the current checkpoints and the output audit directories. The companion evidence JSON records configurations, full metrics and hashes. Recognition weights, training data and deployment launchers were not changed to create this report.

The original report's Python 3.11 verification had 114 passing tests with 13 warnings in 30.92 seconds. The Python 3.14 migration result is recorded above. The warnings concern the existing PyTorch tracing/export paths. A passing test suite establishes that the covered program behavior works under those test conditions. It is not a measurement of real users' recognition accuracy.

2. What the system currently supports

Capability	Verified interpretation
Hand tracking	Camera frames can be processed into up to two hands, each with 21 estimated landmark points.
Static classification	The default static checkpoint predicts A-I and K-Y: 24 labels excluding J and Z.
Dynamic classification	The default sequence checkpoint predicts J and Z from time-ordered two-hand landmark slots.
Static collection	The guided static studio records labeled sessions, processes them and creates versioned training runs.
Image import	The extractor reads supported still images from folders or ZIPs, extracts usable single-hand samples and creates audit records.
Dynamic collection	Dynamic Studio and CLI tools save landmark clips for movement training.
Browser support	TypeScript helpers and model-export tooling exist. A complete deployed browser recognition application is not established by these files.

A class is one output category, such as M. Supporting a label in software is different from having enough representative training and test examples to recognize it reliably. README examples such as HELLO or THANK_YOU describe possible future datasets; the retained dynamic checkpoint has only J and Z.

The system does not currently establish sentence understanding, grammar, unrestricted sign vocabulary, reliable unknown-sign rejection or body-and-face interpretation. Those require additional data, input features, models and evaluation. Sign names and acceptable variants also need a language-specific definition and knowledgeable human review.

3. End-to-end architecture

The live path is: camera frame -> hand detector -> landmark preparation -> static or dynamic classifier -> confidence and timing decisions -> displayed or emitted sign.

OpenCV handles camera images. MediaPipe's pretrained detector estimates hand locations, landmark coordinates and handedness. The project's detector adds tracking-related protections such as duplicate-hand suppression and handedness stabilization. Landmark preprocessing converts those points into the exact numerical form expected by the selected checkpoint. Inference means running a trained model to obtain predictions, without updating its weights.

The static branch describes handshape in one frame. The dynamic branch maintains a time window and describes changing handshape and travel. Motion routing decides when a moving hand should use the sequence branch instead of producing static letters during transitions. A stabilizer then waits for repeated evidence and controls duplicate emissions.

Component	Main implementation	Responsibility
Capture and geometry	asignx/camera.py, detector.py, models.py	Acquire frames and represent detected hands.
Static recognition	ml/preprocessing.py, model.py, predict.py	Prepare one hand, score 24 poses and stabilize decisions.
Movement recognition	ml/video.py, sequence_model.py, predict_sequence.py	Prepare clips, buffer samples and classify movement.
Dataset management	ml/extract_dataset.py, data_io.py, sequence_io.py, manual_recording.py	Read, validate, collect and preserve data.
Training and evaluation	ml/train.py, train_sequence.py, evaluate.py, benchmark.py	Fit networks, select checkpoints and compute metrics.
User tools	main.py, extractor_gui.py, manual_training_gui.py, dynamic_studio_gui.py	Recognition and data-collection workflows.
Browser helpers	web_client/asignx_client_sdk.ts, ml/export_onnx.py	Client feature preparation, feedback and export.

Keeping detection, classification and presentation separate makes failures easier to diagnose. A missed hand is a detector problem; a correctly detected M predicted as N is a classifier or label problem; a correct label emitted twice is a state-management problem.

4. Landmark data and what the representation loses

A landmark is an estimated anatomical point, such as a wrist, fingertip or finger joint. Each detected hand has 21 points and each point has x, y and z coordinates. One hand therefore produces 21 x 3 = 63 numbers. These are estimates from images, not direct physical measurements of the bones. The depth coordinate should not be interpreted as calibrated physical distance merely because the array has three components.

The static CSV has 67 columns: label, session_id, handedness, preprocessing and 63 coordinates in the order x0,y0,z0,x1,y1,z1 through x20,y20,z20. A float32 is a numerical value stored with 32-bit floating-point precision. Consistent ordering and numerical precision matter when reproducing preprocessing and comparing extracted samples.

The coordinates retain a useful description of handshape while discarding image texture, skin appearance and much of the background. This is a compact input for a small classifier. However, a static wrist-relative sample also loses the hand's absolute position and its path through time. The CSV does not inherently retain the original image path or verified signer identity. Import audits can preserve those links for new imports, but cannot manufacture missing historical provenance.

The dynamic NPZ format stores an array of frames and features, its label, session, handedness policy, preprocessing version and optional JSON metadata. NPZ is a compressed NumPy array container. For the retained model, one input is 30 frames x 126 values: two 63-value hand slots in stable Left-then-Right order.

Two-hand slots describe hand relationships, but are not an arm or body model. Missing hands need consistent handling; otherwise an absence can be confused with a real geometric value. More input dimensions do not create information that the collector never measured.

5. Preprocessing, normalization and handedness

Preprocessing is the transformation from detector output into model input. A network trained with one transformation must receive the same transformation during recognition. A silent mismatch can make an otherwise working model fail.

For static poses, the code subtracts the wrist coordinate from every point. This makes the wrist the origin, or numerical zero. It then divides all coordinates by the largest wrist-to-landmark distance. The farthest point becomes one unit from the wrist. This normalization reduces sensitivity to where the hand appears and its apparent size, while preserving much of its orientation.

In compact notation, normalized_point = (point - wrist) / maximum_hand_radius. The radius uses Euclidean distance, the ordinary straight-line distance obtained from the x, y and z differences. Degenerate geometry, where all points coincide, cannot be divided safely and is rejected. Nonfinite values such as NaN and infinity are also rejected.

The base static contract is wrist_max_radius_v1. The retained classifiers additionally expect left_x_reflection_v1: reverse the sign of the x coordinate for a Left hand while leaving y and z unchanged. This is canonicalization, meaning a consistent convention for equivalent left/right inputs. It does not rotate the palm into a universal viewing direction and does not guarantee that every sign preserves meaning when performed with the other hand.

The CSV retains its original hand convention. Canonicalization is applied when preparing classifier inputs. Applying the reflection twice reverses the intended effect. Browser CSS mirroring is only a display change unless the pixels sent to the detector are also changed; handedness and input coordinates must refer to the same image convention.

Dynamic preprocessing uses sequence_anchor_trajectory_v2. It anchors the clip at the earliest visible wrist and uses a shared median hand-radius scale, preserving whole-hand movement and relative placement. Independently centering every dynamic frame at its own wrist would erase the travel needed for movement recognition. Interpolation resamples a clip to a fixed length by estimating values between recorded times; it does not generate new independent performances.

6. Static model: a kinematic residual MLP

MLP means multilayer perceptron: a neural network built from layers that combine all incoming numerical features. The project's own static sign classifier is an MLP, not an image CNN. CNN means convolutional neural network; image CNNs learn spatial patterns directly from pixels. Here, MediaPipe handles the upstream image interpretation, and ASignX's classifier operates on coordinates.

Kinematic means describing geometry and movement without modeling the forces that caused them. The feature enricher adds 28 geometric measurements to the original 63 coordinates: 15 finger-joint flexion cosines, 10 fingertip-pair distances and three components of the palm normal. A cosine describes the angle between two directions; a palm normal is a direction perpendicular to the plane formed by the wrist, index-base and pinky-base points. The resulting vector has 91 features.

Stage	Shape or operation	Plain-language purpose
Input	63 coordinates	Describe the normalized hand.
Enrichment	63 -> 91	Make useful bending, spacing and orientation measurements explicit.
First stage	91 -> 128; normalization, activation, dropout	Learn combinations of those measurements.
Residual block	128 -> 128	Refine the representation while preserving a direct path from the input.
Second stage	128 -> 64; another residual block	Compress the pose into a learned summary.
Classifier head	64 -> 24 logits	Compare the summary with one learned vector per class.

A Linear layer computes weighted sums. LayerNorm, or layer normalization, rescales a sample's internal activations to make optimization more manageable. GELU is a smooth activation function: it allows nonlinear relationships rather than only straight-line combinations. Dropout temporarily removes selected internal values during training to discourage reliance on a narrow set of features; it is disabled for ordinary inference.

A residual connection adds a block's input back to its transformed output. An embedding is the 64-number summary learned before the final classifier. The cosine-prototype head compares the embedding's direction with learned class vectors, then applies a learned scale. A logit is a raw class score before conversion to probabilities. The default model has 63,681 trainable parameters. The manual model uses the same architecture and parameter count, with different learned weights.

Prototype means a representative vector for a class. A learned prototype in the embedding space differs from the 63-coordinate class reference used for finger coaching. They are not interchangeable data structures.

7. Dynamic model: temporal convolution and attention

The retained dynamic model has 296,641 parameters and labels J and Z. Although its compatibility wrapper is named SignLSTM, the saved configuration selects backbone=tcn_attention. A backbone is the central feature-processing network. Reading the class name alone would incorrectly describe the saved model as an LSTM.

The input contains 30 frames and 126 coordinate values per frame. Kinematic enrichment expands each hand from 63 to 91 values, producing 182 values per frame. A learned projection maps them to width 128. Two temporal residual blocks use dilations 1 and 2. Four-head self-attention combines information across the window. Mean pooling averages the resulting time-step representations, and a projection produces a 64-number embedding for the two-class cosine head.

Temporal means relating to time. A TCN, or temporal convolutional network, slides learned filters along a sequence rather than an image. Dilation spaces the sampled time positions farther apart, allowing a small filter to cover a wider time range. Self-attention lets one time position combine information from other positions in the same clip. Four heads means four parallel learned attention comparisons, not four separately trained sign classifiers.

Pooling reduces several time-step outputs to one clip summary. The loaded SignLSTM wrapper uses mean pooling as recorded in its checkpoint. The separate SignTCNAttention class in the source has an attention-weighted pooling implementation; that alternative must not be confused with the exact retained checkpoint path.

An LSTM, or long short-term memory network, is a recurrent model that carries a learned internal state through a sequence. It remains an available alternative in this codebase. Its existence does not mean the deployed dynamic checkpoint uses it.

Thirty steps at the predictor's 15 samples-per-second default represent roughly a two-second window. Exact timing depends on sampling and window handling. The older checkpoint does not record every newer timing field; the current predictor falls back to 15 FPS when sampling_fps is absent. FPS means frames, or samples, per second. Recognition should use timestamp-based sampling so that a faster camera does not silently shorten the gesture window.

8. How training works

Supervised learning uses inputs paired with target labels. For a static sample, the input is a prepared hand vector and the target is a letter. During training, the model predicts scores, computes a loss measuring disagreement with the target, calculates gradients and adjusts its weights. A gradient describes how changing a weight would change the loss. Backpropagation is the procedure used to calculate those gradients through the network.

An epoch is one pass through the training data. A batch is the group of samples processed before a weight update. The learning rate controls the size of updates. AdamW is the optimizer used here: it adapts updates using gradient statistics and applies decoupled weight decay. Weight decay discourages excessively large weights. These settings influence fitting; none guarantees good recognition on unfamiliar people.

Cross-entropy loss penalizes putting too little probability on the target class. Label smoothing reduces the target's absolute certainty slightly, which can reduce overconfidence in training labels. Data augmentation adds controlled variations, such as small 3D rotations and coordinate jitter. Jitter is small random perturbation. Augmented versions of one recording remain dependent on that original recording and must stay in its split.

The default static checkpoint records 50 epochs, batch size 64, learning rate 0.002, weight decay 0.0001, label smoothing 0.05, dropout 0.2, augmentation, seed 42 and CPU training. The saved selected epoch is 25. A random seed initializes the pseudo-random choices for reproducibility; identical seeds do not guarantee identical results across all hardware and library versions. Older metadata omit some options added later, so today's defaults cannot be assumed to describe every detail of that historical run.

The current trainer supports validation-macro-F1 checkpoint selection, validation-loss tie-breaking, class weighting and cosine learning-rate decay. Class weighting gives underrepresented labels more influence on the loss; inverse weighting uses the reciprocal of their training frequency. Cosine decay gradually reduces the learning rate following a cosine-shaped schedule. Selection chooses which saved epoch to retain; it is different from the optimizer's per-batch updates.

The new manual run records inverse class weighting, macro-F1 selection, augmentation scale 1.0, no global group plan and test evaluation enabled. It completed 50 epochs and selected epoch 16. The dynamic checkpoint records batch size 16, learning rate 0.0015, 50 epochs and selected epoch 14. The report inspects these records rather than retraining either model.

9. Current dataset inventory and today's changes

Measure	Current inspected state
Static rows	159,534
Static classes	24
Session identifiers	1,648
Handedness	Left 62,018; Right 97,516
CSV columns	67
Stored preprocessing	wrist_max_radius_v1 for all rows
Manual-run training / validation / test	111,792 / 21,533 / 26,209
Earlier cleaned snapshot	157,734 rows; 1,639 sessions; M count 3,388
Today's difference	+1,800 rows, all M and Left; +9 session identifiers

The frozen CSV inside ml/models/manual_runs/20261022T071741Z-87dde74ec0df matches the current main CSV by SHA-256. That establishes which dataset the manual run used. The run directory preserves its checkpoint, metrics, split manifest and training log. Its timestamp is an artifact identifier in UTC; it is not a separate local-date interpretation.

Class	Rows	Class	Rows	Class	Rows
A	5,853	I	7,037	R	7,451
B	5,869	K	7,998	S	6,241
C	6,661	L	8,252	T	7,106
D	7,086	M	5,188	U	6,876
E	6,089	N	5,281	V	7,410
F	7,372	O	6,221	W	6,041
G	7,534	P	5,538	X	7,000
H	7,506	Q	5,557	Y	6,367

A sample is one landmark record, not necessarily an independent person or gesture. Two hundred adjacent frames of one held pose can provide much less diversity than a smaller number of performances by different people. The nine new session identifiers therefore must not be described as nine new signers. Their true person identities require collection records.

The new samples increase M's count from 3,388 to 5,188. They do not by themselves prove that the earlier M/N confusion is solved. The historical weak-class dataset and combined CSV are separate experiment artifacts and must not be silently counted as active training data.

10. Splitting, leakage and benchmark changes

Training data update the model. Validation data guide choices such as the best epoch, thresholds or architecture. Test data estimate performance after those choices. A held-out set is kept outside the fitting process. Repeatedly using its results to choose the next change turns it into development evidence, even if no gradients are computed on it.

Data leakage occurs when information that should be excluded from evaluation enters training or selection. Near-identical frames, transformed copies and recordings from the same person are common sources. Session separation prevents reuse of the exact group string; signer separation keeps a real person out of other partitions across every class. A session ID is not proof of a person's identity.

The current validators check index coverage, label coverage, session separation, exact canonical-feature leakage and conflicting exact labels. They cannot identify every near-duplicate or recover hidden shared signers. The optional global group-plan tools can enforce supplied identities across labels, but the manual model records group_plan=null. Its benchmark is therefore session based, with signer independence unverified.

Adding data and regenerating a split can move existing groups. The manual run has 111,792 training rows, 21,533 validation rows and 26,209 test rows. The historical canonical checkpoint was trained with 109,223 training rows, 13,791 validation rows and 36,973 test rows. Using the same seed did not preserve those assignments after the dataset changed.

New partition	From old train sessions	From old validation sessions	From old test sessions
train	107,522	784	1,886
validation	754	12,651	8,128
test	947	356	24,706

The new test's 947 rows from old training sessions and 356 from old validation sessions make it unsuitable as a wholly unseen comparison set for the canonical model. Conversely, the new training partition includes 1,886 rows from historical test sessions. Evaluating the manual model on the old test without checking these relationships would also overstate independence.

The appropriate next benchmark should freeze verified signer/source groups before any model is trained. Compare candidates on the same held-out groups, tune on validation only and preserve a final untouched population for release assessment. Grouped evaluation guidance explains why the group must match the intended unseen population; it cannot supply missing identities. [W3]

11. Extraction and cleaning workflow

The extractor accepts JPG, JPEG, PNG, BMP and WebP still images from a folder or ZIP. The static format holds one hand, so multi-hand images are skipped. Known videos and GIFs are rejected, and animated image handling is checked during reading. An image file can still depict a sign that requires movement; the file format alone cannot establish that its label belongs in a static dataset.

The user selects the input and destination CSV, previews inferred sign names and supplies a sign-names CSV when the folder layout is ambiguous. The optional manifest supplies each relative image path, label and origin group. Labels inferred from folder names must be checked against the intended sign vocabulary.

Gentle cleaning removes unreadable images, unusable landmarks and exact repeats while preserving pose variation. Stronger settings also reduce very similar poses or remove statistical outliers. An outlier is an example far from the common pattern. It might be a mistake, or a correct rare viewpoint; unusual does not automatically mean wrong. Aggressive cleaning should therefore be justified by source review.

The importer records source paths, image hashes, grouping, output row references and decisions in audit JSON. Provenance means the origin and processing history of a record. Exact copies assigned to different labels are skipped and recorded for review instead of choosing a label automatically. Source images remain unchanged, and existing accepted rows are preserved during appends.

Source groups should be stable across imports and devices. Renaming a recording or copying it to another folder must not create artificial independence. New audit records improve future traceability, while historical rows may remain unresolved.

The reviewed Windows package is under output/extractor-reviewed, with its matching distribution ZIP. The launcher uses that package and falls back to source. A retained executable is a snapshot of its build-time code; source changes do not update an already-built executable. Package identity and preprocessing should be checked before comparing extraction behavior.

ZIP path checks and a 10 GiB total uncompressed-size limit remain. ZIP entry count is not capped in the current guide. CSV replacement uses locking and changed-file checks, but the guide documents a remaining race with external writers that do not share the lock. Data-writing tools should not concurrently edit the same CSV.

12. Source recovery and the M corrections

The source investigation tied 24 hashed import sessions, historically 96,145 rows, to archive (4).zip. These are archive-origin groups, not verified signer identities. In the original benchmark, some labels from that archive appeared in training while G, I, K, M, O, Q and X from the same archive were assigned to testing. This is a source/class coverage gap: a model can see a source's A examples without seeing its G examples.

The M folder contained 7,900 image filenames representing 4,876 distinct image contents. A geometric recovery linked original CSV row 110182 to M (100).jpg. The owner then identified the downward three-extended-fingers pose for exclusion. The first correction removed one record; the subsequent reviewed pass removed another 2,252, for 2,253 removed landmark rows in total.

The matching rule required all 63 canonical float32 coordinates to agree within absolute tolerance 0.000001, with no relative tolerance, and required the verified M source session. A tolerance is the permitted numerical difference when comparing extracted values. This is stronger evidence than deleting the nearest-looking landmark vector or trusting the classifier's prediction as the label.

The removal evidence links 4,083 source filenames to removed rows; filename count and sample count differ because identical image content has multiple names. After that correction, 1,062 archive-4 M rows remained: 855 linked to retained images and 207 that could not be reliably linked. The unresolved rows were preserved. These records establish an M-specific exclusion; they do not establish a separate N-class deletion.

The reviewed snapshot had 157,734 rows and SHA-256 beginning 4a160bec7355. Today's additions produce a different 159,534-row file. The latest-correction JSON remains a historical correction record and should not be interpreted as a live inventory of the current main CSV.

Source ZIPs, correction backups, row remaps, contact sheets and unresolved-row records remain evidence. Original and current row numbers differ after deletions. A higher score after removing disputed test labels changes the evaluation population; it does not show that unchanged weights learned anything new.

13. Understanding recognition metrics

Accuracy is the fraction of all evaluated samples classified correctly. Precision asks: of the samples predicted as M, how many really were M? Recall asks: of all real M samples, how many did the model recover? Support is the number of evaluated examples belonging to a class.

F1 combines precision and recall using the harmonic mean: F1 = 2 x precision x recall / (precision + recall). For example, if the model predicts M 100 times and 80 are correct, precision is 80%. If there were 160 actual M examples, recall is 50%. M's F1 is approximately 61.54%. This example illustrates the calculation and is not an ASignX measured result.

Macro F1 averages class F1 values equally. Weighted F1 gives classes influence in proportion to their sample counts. In an imbalanced test set, a model can score well on many small classes while missing many examples from a few large classes. This helps explain why the manual model's macro F1 is 74.23% while its accuracy is 59.32%. Neither metric should replace per-class inspection.

A confusion matrix counts actual labels against predicted labels. Its diagonal contains correct predictions; other cells show specific confusions. A high-recall, low-precision class attracts too many incorrect predictions. A high-precision, low-recall class is predicted conservatively but misses many true examples.

Softmax converts logits into nonnegative values summing to one over the known classes. Those values are commonly called confidence or probability, but a score of 0.8 is not automatically an 80% real-world chance of correctness. Calibration measures and adjusts how well confidence matches observed correctness. An unfamiliar sign can still receive a high softmax score because the model must distribute probability among the known labels.

Thresholded evaluation rejects scores below a chosen cutoff. Accepted-only accuracy excludes rejected examples and can look impressive while the system rejects most useful attempts. Reports should also count missed known signs, acceptance coverage and false acceptance of unknown inputs. Offline landmark classification starts after usable landmarks already exist; live end-to-end evaluation must also count detector failures and timing mistakes.

14. Current static results and their interpretation

The table below uses fresh unthresholded evaluations on the exact frozen manual-run validation and test partitions. Unthresholded means taking the highest-scoring class for every usable input. No runtime confidence rejection, camera acquisition or temporal stabilization is included.

Model / partition	Samples	Accuracy	Macro F1	Weighted F1
Canonical default / validation	21,533	77.60%	85.29%	79.96%
Canonical default / test	26,209	61.79%	74.84%	62.31%
Manual run / validation	21,533	77.17%	85.79%	79.62%
Manual run / test	26,209	59.32%	74.23%	60.03%

The manual checkpoint's reproduced values agree with its saved validation and test results. This verifies the report's reading of that run. The canonical model's 74.84% test macro F1 exceeds the manual model's 74.23% by about 0.61 percentage points on these rows, while its accuracy is about 2.48 points higher. The manual model is slightly ahead on validation macro F1. These are observed differences, without a significance claim.

Because these partitions overlap with the canonical model's earlier training and validation sessions, the comparison cannot justify promoting or rejecting a model on unseen-user performance. It also cannot establish that today's M additions caused an improvement. A controlled before/after study would hold the benchmark fixed and change only the training input being investigated.

Historical measurements remain useful when labeled correctly:

Model / population	Validation macro F1	Test macro F1	Test accuracy
Canonical / original test, 36,973 rows	97.33%	59.49%	53.19%
Canonical / cleaned test, 34,720 rows	97.33%	63.07%	56.62%
Earlier retrained candidate / cleaned test	97.72%	57.77%	49.57%
Manual run / current test, 26,209 rows	85.79%	74.23%	59.32%

The first two rows use identical canonical weights on different test populations. The earlier candidate used preserved historical partitions; the current manual run uses a regenerated partition. This table is a benchmark history, not a chronological learning curve. The large historical validation-to-test gap supports investigating source mismatch and labels; it does not identify a single proven root cause.

15. Per-class results and targeted diagnosis

The following results describe the manual checkpoint on its current 26,209-row test partition. They must not be mixed with the earlier cleaned-test per-class table, whose support counts and sample assignments differ.

Class	Support	Precision	Recall	F1
A	235	78.19%	99.15%	87.43%
B	237	94.05%	100.00%	96.93%
C	431	93.19%	98.38%	95.71%

Class	Support	Precision	Recall	F1
D	472	31.29%	99.58%	47.62%
E	355	81.03%	92.68%	86.47%
F	387	97.73%	100.00%	98.85%
G	4,604	100.00%	11.84%	21.17%
H	515	13.31%	99.81%	23.48%
I	4,373	99.78%	93.05%	96.30%
K	4,999	94.53%	34.55%	50.60%
L	558	48.72%	99.28%	65.37%
M	556	95.32%	98.92%	97.09%
N	394	92.16%	98.48%	95.21%
O	313	89.68%	100.00%	94.56%
P	318	87.86%	95.60%	91.57%
Q	317	98.73%	98.42%	98.58%
R	520	28.88%	95.96%	44.40%
S	354	44.81%	100.00%	61.89%
T	439	55.79%	96.58%	70.73%
U	397	85.08%	96.22%	90.31%
V	515	14.15%	99.61%	24.78%
W	274	99.63%	99.27%	99.45%
X	4,333	98.29%	33.07%	49.49%
Y	313	87.68%	100.00%	93.43%

The current manual-test lowest F1 classes are G (21.17%), H (23.48%), V (24.78%), R (44.40%), D (47.62%). The largest current confusion counts are G predicted as H: 3,239, K predicted as V: 3,083, X predicted as R: 1,093, X predicted as D: 942, G predicted as L: 566.

For weak classes, inspect original images or recordings alongside detector geometry, handedness, predictions and source group. First establish whether the intended label is correct; then determine whether the detector preserved the relevant distinction. Collect additional legitimate examples of the observed confusion under varied people and conditions. A model's error is evidence for investigation, not sufficient evidence for deletion or relabeling.

The historical cleaned test's weakest F1 values included P, G, V, D and H. A different weak-class ordering in the current test can reflect moved source groups as well as changed weights. Track the actual samples and confusions when deciding which data to collect.

16. Dynamic results and shortcut learning

The retained movement checkpoint's recovered benchmark contains only 11 clips: seven training, two validation and two test. Its two test clips, one J and one Z, both classify correctly, yielding 100% test accuracy and macro F1. This is a tiny benchmark, not proof of reliable recognition across people, devices and movement styles.

Historical temporal diagnostics found that the original two-clip test remained at 100% after time reversal, frame shuffling, repetition of the final frame or removal of wrist travel. An ablation is a diagnostic experiment that removes or alters part of the input to test what the model depends on. These results show that this benchmark does not require correct time ordering for success. They do not prove that the network never uses motion on other data.

Shortcut learning means exploiting an easier correlation, such as a distinctive ending pose, instead of the intended concept, such as the correct J trajectory. Static handshape differences can make J and Z easy to separate without understanding their paths. The architecture's ability to model time is not evidence that a trained model actually learned the desired movement.

Build a broader benchmark containing varied signers and speeds, both hands as appropriate, camera changes, complete gestures, holds with matching handshape, incorrect movement paths, idle movements and transitions. Hard negatives are deliberately confusing non-target examples. They test whether the system distinguishes a valid sign from something that merely resembles part of it.

Continuous-stream evaluation should count missed signs, false events, duplicate events, repeated-letter handling and detection-to-emission delay. Correctly classifying isolated clips with known boundaries is a simpler task than deciding when a gesture starts and ends in a live stream. Synthetic software tests and five additional J clips do not fill missing Z, negative or signer coverage.

17. Live confidence, motion routing and stabilization

The combined desktop CLI defaults to confidence 0.8 for both branches, five stable static frames, ten release frames and motion threshold 0.045. These are decision settings, separate from the unthresholded metrics in this report. The standalone sequence predictor uses its own window-based stabilization settings; frames and windows are different units.

Stabilization waits for repeated compatible predictions to reduce flicker. Hysteresis means using different conditions to enter and leave a recognized state, avoiding rapid switching around a cutoff. Release handling allows the same sign to be emitted again after the hand leaves or the user returns to a nonaccepted state. These decisions affect usability even if the per-frame classifier remains unchanged.

Motion routing uses wrist and fingertip movement over elapsed time. Relying only on the wrist can miss finger movement; relying on distance per frame makes behavior change with camera rate. Python and TypeScript include elapsed-time logic with a reference 30-FPS scale. The threshold is expressed in that implementation's normalized motion score, not meters per second.

Sequence sampling tracks timestamps, rejects poor windows and resets after stalls or tracking gaps. The combined route also considers motion and a short period after movement. These protections reduce stale or inappropriate predictions but still need end-to-end testing with real transitions.

Frame-count settings have a time cost. Five processed frames at 30 FPS span about 0.17 seconds and ten about 0.33 seconds; at 15 FPS those counts span about twice as long. These are illustrative count-to-time conversions, not measured application latency. A target device's detector speed and processing schedule determine the actual delay.

Historical validation calibration fitted a threshold of 0.535, but that setting did not outperform the existing 0.8 threshold on the historical test. The default remains 0.8. Unknown examples were insufficient to establish dependable rejection, so confidence calibration should not be described as a completed unknown-sign solution.

18. ASignLearn: coaching and interpretation

ASignLearn is a guided-attempt workflow: the application already knows which sign the learner is trying to perform. The TypeScript ASignLearnCoach computes that target's softmax score, accumulates hold progress and compares each finger with a class reference. Its defaults are 15 hold frames, pass confidence 0.75 and finger-error tolerance 0.14 in normalized coordinate units.

The coaching reference is a coordinate-wise median hand vector computed from training examples. Although exported as class_centroids, it is not the same as the learned cosine classifier prototype. A centroid usually means an average representative point; here the actual implementation uses medians for the 63-coordinate reference. Finger error averages landmark distances for the joints of that finger.

This is a geometric suggestion system. A finger close to the reference may still be wrong for a meaningful sign variant, while a valid different anatomy or viewpoint can be farther away. It does not establish teaching effectiveness or a linguistically complete explanation of the mistake. Knowledgeable sign review should validate reference poses and wording.

A concrete current issue is that missing centroid data leave meanError at zero in evaluateAttempt. The coach can therefore report that every finger is aligned even though no reference was available. The product should represent that case as unavailable feedback. This is a report finding; the SDK was not edited as part of writing the report.

Hold progress increases above the target threshold and decreases by one below it; it is not necessarily a strictly consecutive hold. The coach's state also belongs to the caller: reset it when changing target or ending an attempt so that progress is not carried between lessons. Per-finger hints currently say to adjust a finger; they do not derive a detailed direction of movement or validated anatomical correction.

19. ASignLive and the current web SDK

ASignLive is an open recognition workflow: the user has not supplied a target sign. It therefore needs stronger unknown rejection, movement routing, release logic and event evaluation than the guided coaching case.

The TypeScript file exports landmark normalization, checkpoint-aware static preparation, 91-feature enrichment, softmax, the coach and the live stabilizer. The live stabilizer accepts already-computed logits and returns per-hand prediction, confidence, stable label, newly emitted label and motion status. It contains no model weights or standalone browser neural-network execution engine.

The file header describes a browser-only pipeline using MediaPipe and ONNX Runtime, but the file itself does not initialize those libraries, request a camera, download assets, run ONNX sessions or implement the complete dynamic sequence predictor. Those operations need an application integration layer. The absence of those operations in this file is why its header is an architectural intent rather than proof of a working deployed site.

The graph exported for the current static model accepts 63 prepared values and performs kinematic enrichment internally. The client must not pass the helper's 91-value output into a 63-input graph. Browser and Python must agree on coordinate order, left-hand reflection, input shape and label order.

The metadata interface declares hidden_dim, while the static exporter writes hidden_sizes. Runtime inference may not need that field, but a typed integration should reconcile the schema rather than casting mismatched JSON without validation. Class labels, preprocessing contract and numerical dimensions should be checked when loading a release.

Use separate stabilizer instances for independently tracked hands. Its update method accepts a hand name but stores a single candidate and motion history internally. Interleaving two hands through one instance would mix state. Also, markHandMissing clears candidate and motion history immediately but clears stableLabel only after the release-frame count; a displayed label can remain briefly during a missing-hand interval. The desired display behavior should be specified and tested.

20. Browser export, performance and deployment

ONNX is an interchange format for trained computation graphs. ONNX Runtime Web executes compatible graphs in the browser. WebAssembly, or WASM, is a portable execution format commonly used for browser CPU work. WebGPU provides browser access to supported GPU computation. Runtime provider and operator support must be tested with the exact exported model and target browser. [W1]

ml/export_onnx.py writes metadata and optional JSON weights, then attempts ONNX export at opset 17. An opset is a versioned set of ONNX operations. If export raises an exception, the code falls back to a traced TorchScript file. TorchScript is a PyTorch execution artifact; that fallback does not establish an ONNX graph that ONNX Runtime Web can load. Since metadata can exist even when ONNX export fails, release checks must inspect the returned artifact type and run an actual browser inference.

A weights JSON serializes tensors. It is not a replacement for an inference engine; code still needs to implement every layer and operation. Rounding and alternate execution paths can change outputs. Compare fixed inputs across Python, the exported graph and the browser before relying on confidence thresholds.

A practical browser path is: camera -> MediaPipe Hand Landmarker -> shared preparation -> ONNX Runtime session -> product-specific state -> UI. MediaPipe's JavaScript detectForVideo operation is synchronous, so worker-based processing should be considered to keep the interface responsive. A worker is a separate browser execution context used to move processing away from the main UI thread. [W2]

Local inference can avoid uploading camera video for recognition. That privacy property depends on the final application's network behavior, asset loading and analytics, not just this helper file. Model/runtime files still need to be delivered and cached. ONNX Runtime's deployment guide describes the runtime resources that need to accompany a web application. [W4]

Benchmark detector time, preprocessing, model execution, window accumulation and UI delay separately. Record median and 95th-percentile latency, memory, startup time, missed-frame rate and sustained performance on target devices. A percentile describes the delay below which a proportion of observations

fall; the 95th percentile exposes slower cases hidden by an average. Small parameter counts suggest a compact classifier, but do not measure the cost of the detector or browser pipeline.

Use WASM as a tested baseline and compare WebGPU on representative devices; GPU scheduling overhead can matter for small models. Pin compatible detector, runtime and model artifacts in a release bundle, and provide explicit camera-permission, unsupported-device and missing-asset behavior. These are deployment recommendations, not completed device measurements.

21. Are hands enough, and which models make sense?

Hands can be sufficient for a constrained alphabet-pose vocabulary where finger configuration and orientation distinguish the labels. Broader signing can depend on where the hand is relative to the face or torso, arm movement, two-hand contact and facial expression. Wrist-centered static coordinates cannot recover those cues once removed. The source declares possible upper-body feature constants, but the inspected retained models still accept hand-only input dimensions.

Adding body or face features requires an explicit new preprocessing contract, synchronized detection, missing-data handling, suitable training recordings and retraining. Attaching extra coordinates to a 63- or 126-input checkpoint is not a compatible upgrade. An upper-body detector may also increase device cost, so the vocabulary should establish which cues are actually necessary.

Model family	Useful role	Main limit or tradeoff
Plain MLP and current residual MLP	Static landmark baselines	Cannot infer absent movement or body context.
SVM or tree classifier	Alternative static baseline	Needs its own calibration and deployment comparison.
LSTM or GRU	Sequence baseline with recurrent state	Requires representative complete motion and timing evaluation.
Compact TCN	Local motion patterns over a window	Needs window and boundary handling.
TCN plus attention	Local and window-wide motion relationships	Additional complexity must earn its cost on a meaningful benchmark.
Spatial-temporal graph network	Explicit skeleton-joint relationships	More engineering and data; not an automatic improvement.
Image CNN or image/landmark hybrid	Visual details missing from landmarks	More compute and potential reliance on source/background shortcuts.

An SVM, or support vector machine, learns decision boundaries between classes. A GRU, or gated recurrent unit, is another recurrent sequence model. A graph network represents joints as connected nodes, reflecting anatomical structure. A hybrid combines different input or model types. These are candidate comparisons, not claims that any one will outperform the current network.

The project-specific recommendation is to resolve benchmark identity and source coverage before expanding model size. Compare a small number of alternatives using the same data, validation policy,

training budget and several seeds. A larger model can fit a deficient benchmark more closely without improving recognition for new users.

22. Scaling the vocabulary and adding signs

The cosine head supports adding a new class prototype from example embeddings. This can be useful for experiments or personalization. Few-shot learning means adapting from a small number of labeled examples; prototype addition is one mechanism, but its presence alone does not demonstrate reliable few-shot recognition.

A new prototype competes with every old class. It can attract inputs previously classified correctly, and the encoder may not preserve a distinction that was absent during its original training. One prototype may also poorly describe several valid viewpoints. Training-free addition therefore requires fresh old-class, new-class and unknown-input evaluation.

Fine-tuning continues training from saved weights. Head-only tuning updates the classifier while keeping the encoder fixed; full tuning can change both. Catastrophic forgetting is the loss of old capabilities while learning new ones. Replay mixes representative old examples into new training. Distillation asks a new model to preserve useful predictions from an earlier model. These are possible safeguards to evaluate, not guarantees.

The registration helper can carry forward earlier dataset and split metadata after labels change. That history does not evaluate the expanded vocabulary. Save a new model version with explicit support examples and a new evaluation card. A fixed ONNX output dimension also cannot grow simply because a Python prototype was added: re-export the model or deliberately implement a separately versioned embedding-and-prototype client.

A scalable release bundle should bind the graph, labels and language definitions, preprocessing, detector version, runtime settings, thresholds, metrics and content hashes. A dataset catalog should preserve source, signer, session and review history. This is a proposed release architecture; a single mutable CSV and familiar checkpoint filename do not supply those guarantees.

23. Current tools, artifacts and development history

Artifact or tool	Current role
ml/models/sign_classifier_canonical.pth	Default static checkpoint selected by the combined launcher.
ml/models/sequence_classifier.pth	Default J/Z movement checkpoint.
ml/models/manual_runs/20261022T071741Z-87dde74ec0df/	Today's frozen CSV, manual candidate, manifest, metrics and training log.
ASignX_TestRecognition.bat	Launches the retained static and sequence defaults.
ASignX_ManualTraining.bat	Opens guided static collection and versioned training.
ASignX_DataExtractor.bat	Opens the reviewed extractor package, with source fallback.
ASignX_DynamicStudio.bat	Opens Dynamic Studio using its executable-first behavior.
output/current-model-comparison/	Historical same-cleaned-test comparison.
output/system-report-2026-10-22/	This report's fresh metrics, hashes and software-test log.

The manual studio copies the dataset into a versioned run directory and checks its hash before training. It selects the new model for that studio's test workflow after training completes. Its camera test does not change the combined launcher's default checkpoint. The inspected launcher still points to the canonical static file.

Earlier controlled experiments tried weighting, augmentation and label-smoothing changes. A short five-variant screen selected a no-smoothing candidate on validation but did not beat the canonical model on the historical test. Later 50-epoch retraining likewise underperformed the retained model on the same cleaned test. Those experiments support the decision to preserve the default at that point; they do not settle every possible architecture or training schedule.

The earlier cleanup removed ten experimental checkpoints and approximately 2.37 GiB of older build/package material. Metrics and audit records were preserved, and active models and important data artifacts were checked. Today's new manual checkpoint was created later, so there are now three inspected checkpoint files under ml/models recursively. Historical statements about two remaining model files should be read in that sequence.

The main CSV and its three new backup files were already present as user-workspace changes when this report began. Report generation leaves them intact. Operational verification did not open the camera or measure a real browser session.

24. Priorities, completion criteria and limits

Priority	Work	Evidence needed to call it complete
1	Freeze a meaningful benchmark	Verified identities, source links, fixed group assignments and a final untouched test population.
2	Review weak labels and source coverage	Original evidence, explicit decisions, retained difficult valid cases and resolved provenance where possible.
3	Compare manual and canonical models fairly	Same unseen signer/source test, per-class results, confidence intervals and known/unknown coverage.
4	Expand movement collection	Diverse J/Z and negative clips; path-sensitive tests and continuous-stream event metrics.
5	Complete browser integration	Real ONNX artifacts, Python/browser parity, per-hand state isolation and target-device measurements.
6	Validate coaching	Reviewed references, unavailable-reference handling and user feedback supported by sign expertise.
7	Publish versioned releases	Bundled contracts, asset hashes, evaluation card, reproducible commands and rollback artifact.

Confidence intervals describe statistical uncertainty around a score. For correlated recordings, uncertainty should be estimated over meaningful groups such as signers, rather than pretending every neighboring frame is independent. A practical pass threshold should be agreed for the actual product and vocabulary before looking at final test results; this report does not invent an acceptable user-error rate.

Immediate engineering findings include missing-reference coaching feedback, metadata schema mismatch, browser model loading and dynamic-buffer integration, per-hand state isolation, and export-fallback detection. These are concrete report findings and proposed fixes. They were not silently implemented during report authoring.

The current evidence supports a functioning, modular research prototype with substantial data tooling and integrity checks. It still leaves uncertain new-signer accuracy, independent model ranking, unknown rejection, movement understanding, live event quality and device performance. The next useful progress measure is stronger evaluation evidence on those questions.

25. Plain-language ML glossary

Term	Meaning in this project
Ablation	Remove or alter part of an input/model to test what affects its output.
Activation	Nonlinear transformation inside a neural network, such as GELU.
Architecture / backbone	The structure of a network / its main feature-processing part.
Attention	Learned combination of information from other sequence positions.
Augmentation	Controlled training variations of existing examples; not new independent data.
Batch / epoch	Samples per update / one pass through the training set.
Calibration	Checking or improving agreement between confidence and observed correctness.
Canonicalization	Transforming equivalent inputs into a consistent convention.
Checkpoint	Saved model weights with reconstruction and training metadata.
Class / label	An output category / its recorded name or target assignment.
Class imbalance	Unequal sample counts among labels.
Confidence threshold	Minimum model score accepted by a decision rule.
Confusion matrix	Table counting actual labels against predicted labels.
Cross-entropy	Training loss that penalizes low probability for the correct class.
Domain shift	Change in source or recording conditions between training and use.
Dropout	Training-time removal of some internal values to reduce narrow reliance.
Embedding	A learned compact numerical description before classification.
Feature / feature vector	One input measurement / a list of input measurements.
Fine-tuning	Continuing training from existing weights.
Generalization	Working on examples from beyond the fitting conditions.
Gradient / backpropagation	Sensitivity of loss to weights / procedure calculating it.
Hard negative	A non-target example deliberately similar to a valid sign.
Hyperparameter	A chosen setting such as learning rate, rather than a learned weight.
Inference	Running a trained model without updating weights.
Kinematics	Geometry and movement descriptions without force modeling.

Term	Meaning in this project
Landmark	An estimated anatomical point.
Leakage	Evaluation information entering fitting or model selection.
Logit / softmax	Raw output score / conversion of scores into a known-label distribution.
Loss / optimizer	Numerical training error / algorithm that updates weights to reduce it.
Macro F1	Equal-weight average of the individual classes' F1 scores.
Manifest	Structured record binding data, configuration or split assignments.
Normalization	Rescaling data to a consistent numerical convention.
Overfitting	Fitting development examples better than unfamiliar examples.
Parameter	A learned adjustable number, such as a neural-network weight.
Pooling	Combining multiple time-step representations into one summary.
Precision / recall	Correctness among class predictions / recovery of true class examples.
Prototype	Representative vector used to compare an input with a class.
Provenance	Record of where data came from and how they were processed.
Regularization	Measures such as dropout or weight decay intended to reduce overfitting.
Residual connection	Direct addition of a block's input to its transformed output.
Session / signer	Recording/import group / actual person; these are different identities.
Support	Number of evaluated examples belonging to a class.
Tensor	Multidimensional numerical array, such as batch x time x features.
Validation / test	Data guiding choices / data assessing the finished choice.
Weighted F1	F1 average weighted by each class's sample count.

26. Reproducibility and evidence index

Current main CSV SHA-256: 03b949d3874b68bf0179c73fdfe918da7973ba89345e3b0d981171cc3da461d2

Historical cleaned CSV SHA-256:
4a160bec73556408c12f9457577c765c89e9f8cf09eae3d658fe39d3064f58cd

Historical original CSV SHA-256:
17249bc657b7ed52f1d6a39f5d3bbbf6b8d2445efdab97f64094df68855647db

ml/models/sign_classifier_canonical.pth SHA-256:
98559a10c553c23d2c6aad227f52416b59ad92483bf4f079cfca5eb512bf5b66

ml/models/manual_runs/20261022T071741Z-87dde74ec0df/sign_classifier.pth SHA-256:
4fb80db8f024a4dd5add686ad31d3914d7d05253b2fb4ce2cc63c9fc34a658f9

ml/models/sequence_classifier.pth SHA-256:
1e2228a5970d18b19a25f62cb76fc560aa4c5e33962a8bc2a362a87309db2ec9

The companion evidence JSON preserves full hashes, model configurations, fresh paired metrics, per-class supports, confusion matrices and cross-benchmark session counts. Scores in sections 14 and 15 were freshly computed without retraining. Historical scores were read from saved artifacts and explicitly retain their original population labels.

The fresh evidence builder is docs/collect_detailed_report_evidence.py. It validates the frozen manual snapshot and manifest, checks that it matches the main CSV, and evaluates both static checkpoints on that explicit split. It uses the canonical model there only for a labeled diagnostic comparison; it does not overwrite the canonical checkpoint's historical dataset or split metadata.

Run the evidence collection with: .\.venv-py314\Scripts\python.exe docs/collect_detailed_report_evidence.py

Run software checks with: .\.venv-py314\Scripts\python.exe -m pytest -q

Build the PDF with: .\.venv-py314\Scripts\python.exe docs/build_detailed_system_report.py

Do not run the canonical model's default evaluation command against a changed main CSV and then suppress its identity error. ml.evaluate correctly expects a dataset, checkpoint and manifest that match. Use the original snapshot for historical reproduction, or an explicitly documented cross-benchmark evaluator as used here. Do not rewrite old metadata to manufacture agreement.

Primary local records: output/current-model-comparison/comparison.json; output/data-repair/latest-correction.json; archive/data-corrections/; output/archive4-M-pose-review/; output/recognition-reevaluation/summary.json; output/f1-review/; output/model-cleanup/; output/system-cleanup/; the manual run directory; and this report's evidence/test directory.

External primary references, checked during report preparation:

[W1] ONNX Runtime Web documentation: https://onnxruntime.ai/docs/tutorials/web/ . Used for browser execution-provider terminology and deployment feasibility, not for an ASignX performance claim.

[W2] Google MediaPipe HandLandmarker JavaScript API: https://developers.google.cn/edge/api/mediapipe/js/tasks-vision.handlandmarker . Used for timestamped synchronous video detection behavior.

[W3] scikit-learn cross-validation guide: https://scikit-learn.org/stable/modules/cross_validation.html . Used for why evaluation groups should separate relevant people or sources.

[W4] ONNX Runtime Web deployment guide: https://onnxruntime.ai/docs/tutorials/web/deploy.html . Used for runtime/model asset deployment requirements.

These sources explain external tools and evaluation principles. Project-specific recommendations are engineering judgments based on the inspected code and measured limitations. They do not promise a particular accuracy improvement.